

PriceCheck - Technologies Utilisees et Architecture Logique

1) Technologies utilisees

Frontend (SPA)

- **TypeScript**: ecriture du code applicatif et des types.
- **Vite**: bundler et serveur de developpement (`vite, npm run dev`).
- **HTML5 + CSS3**: templates et styles globaux (`src/style.css`).
- **Application Single Page (SPA)**: rendu dynamique dans `#app` (pas de rechargement de page).

Routing et navigation

- **Router personnalise** (fichier `src/router.ts`) qui mappe un nom de page vers:
 - une fonction `render()` (template HTML)
 - une fonction `init()` (attache les evenements)
- Gestion des pages proteges (auth) et de la page admin (role admin).
- Historique de navigation pour les boutons **Retour** / **Avancer**.

Donnees et stockage

- **Model "database.ts"**: donnees produits et magasins + fonctions de recherche/lecture.
- **Persistence localStorage**:
 - `src/app-db.ts` encapsule `getDbValue/setDbJson/...`
 - `src/admin-data.ts` persiste les modifications admin (produits, imageUrl, prix, magasins)
 - `src/orders.ts` persiste les commandes (`pc_orders`) et la derniere commande (`pc_last_order_id`)

Scanner de code-barres

- **Camera et upload fichier** via les APIs web (`navigator.mediaDevices.getUserMedia, FileReader`) dans `src/scanner.ts`.
- La detection est actuellement **simulee** (fonction `simulateScan` + delais) plutot que via une librairie de scan.
- La dependance quagga est presente dans `package.json`, mais n'est pas referencee dans le code `src/` actuel.

Cartographie

- **Leaflet** charge dynamiquement dans `src/tracking.ts`.
- Tiles **OpenStreetMap** via `https://{s}.tile.openstreetmap.org/....`

Images produits

- `src/barcode-generator.ts` genere une image produit:
 - en priorite via `product.imageUrl`
 - sinon via `fallback` (categorie / Open Food Facts selon barcode)

PWA

- Logique d'installation (prompt) geree dans `src/pwa-install.ts` via l'event `beforeinstallprompt`.

Auth (mode demo / local)

- **Authentification simple localStorage** via `src/auth.ts`:
 - session et role admin stockes dans `localStorage` (`pc_session`, `pc_role`)

2) Architecture logique de l'application

Vue d'ensemble (decomposition)

- **Point d'entree:** `src/main.ts`
 - démarre `navigateTo('home')`
- **Router:** `src/router.ts`
 - centralise la navigation et les gardes (pages proteges, admin)
 - injecte le HTML dans `#app` puis appelle `init()` de la page
- **Templates:** `src/pages.ts`
 - contient les rendus HTML des pages (home, scanner, admin, etc.)
- **Modules par fonctionnalite:**
 - `home.ts`: accueil et mise en panier (selection produit/store)
 - `catalog.ts`: catalogue et comparaison de prix
 - `scanner.ts`: scan et creation de la selection
 - `cart.ts`: panier (calcul total, frais livraison selon mode)
 - `checkout.ts`: paiement + creation de commande
 - `confirmation.ts`: recap / facture (local)
 - `tracking.ts`: suivi commande (polling `localStorage`)
 - `promotions.ts`: gestion/affichage promotions
 - `wallet.ts`: portefeuille/fidelite (contenu UI)
 - `profile.ts`: profil + acces admin
 - `admin.ts`: interface admin + actions (produits, prix, magasins, commandes)

Couche donnees (data layer)

- `src/database.ts`
 - PRODUCTS: liste des produits (imageUrl + prix par magasin)
 - STORES: liste des supermarchés (deliveryFee, etc.)
 - fonctions de recherche/lecture (`getAllProducts`, `searchProducts`, `getStoresByCity`, ...)
- `src/app-db.ts`
 - wrappers `localStorage` pour lire/ écrire JSON
- `src/admin-data.ts`
 - ajoute/modifie supprime/ajuste produits et magasins, puis persiste
 - ex: `updateProductImage`, `removeStore`, `addStore`, `addProduct`, `addOrUpdatePrice`
- `src/orders.ts`
 - `createOrderFromSelection()` -> crée une commande en pending
 - `updateOrderStatus()` -> passe pending vers confirmed
 - `getPendingOrders()` -> liste des commandes en attente (pour admin)

Flux logique de commande

1. Selection du produit + mode (livraison ou sur place)
2. Sauvegarde via `saveCheckoutSelection()` (`orders.ts`)
3. Panier: affichage prix et livraison selon selection
4. Checkout: paiement -> `createOrderFromSelection()`
5. Confirmation: montre numero commande + statut
6. Admin: Valider une commande -> `updateOrderStatus(status='confirmed')`
7. Tracking: polling `localStorage` -> mise a jour UI jusqu'à confirmation